

Sakum Lang

Sakum Lang is a **Hinglish-keyword** (romanized Sanskrit, typeable ASCII) systems language with a self-aware engine, built-in scientific/quantum core, binary-hash query engine, self-rewriting `self` library, and a creator-owned hash key (sutra). Implemented as **raw machine-level assembly** (x86-64 / ARM64 / RISC-V) — there is **no Python, bash, or any other host-language layer anywhere in the project** (per `SAKUM_LANG.md §2`). The core emits native code and portable WASM binaries. Keyword spelling is defined in `SAKUM_HINGLISH.md`.

Build & run (native toolchain only)

```
gcc -arch x86_64 assembly/sakum_simd.s -o /tmp/simd && /tmp/simd      # AVX2 SIMD
demo
gcc -arch x86_64 assembly/sakum_eval.s -o /tmp/eval && /tmp/eval
# self-hosted front end
gcc -arch x86_64 assembly/sakum_self.s -o /tmp/self && /tmp/self      # self-
growing code buffer
gcc -arch x86_64 assembly/sakum_bramann.s -o /tmp/bra && /tmp/bra
# ब्रह्म crawler + scraper
gcc -arch x86_64 assembly/sakum_webhook.s -o /tmp/wh && /tmp/wh      # from-
scratch asm webhook
gcc -arch x86_64 -DSAKUM_LIB_NO_MAIN assembly/sakum_lib_time.s \
    -DSAKUM_LIB_NO_MAIN assembly/sakum_lib_dns.s \
    assembly/sakum_speedtest.s -o /tmp/sp && /tmp/sp example.com      # website
speed tester
gcc -arch x86_64 assembly/sakum_wasm.s -o /tmp/wasmgen && /tmp/wasmgen > /tmp/out.was
m
wasm-validate /tmp/out.wasm      # check the
emitted WASM
node -e "WebAssembly.instantiate(require('fs').readFileSync('/tmp/
out.wasm')).then(x=>console.log(x.instance.exports.run()))"
```

All artifacts are machine code or binary (`.wasm`). SIMD (AVX2 / AVX-512 / NEON / RVV) and quantum-circuit binaries (QCB1) are first-class. See `SAKUM_LANG.md §1.2` and `§1.4`, and `assembly/README.md` for the full machine-level core.

Install your encryption key (सूत्र)

No SHA is used. Provide your own key (the assembly core reads it via the OS):

```
export SAKUM_SUTRA_KEY="your-own-key-here"
# or write it (git-ignored) to sakum_key.txt
```

Layout

```

assembly/      raw x86-64 machine-level core (simd, eval, wasm, self, ...)
examples/      sample .sak programs
                math100.sak  - 100 advanced-math examples
                selflearn100.sak - 100 error-explain / self-learn / bug-resolve example
s
self/          self engine patches / memory ledger
query_logs/    binary-hash query observations
Knowledge/     binary-hash-addressable knowledge tree (sciences + engineering)
research.md    ब्रम्ह (crawler) research log - what it learned from each sphere
upgrade.md     what the crawler/self engine improved in its own core
update.md      live self-update cycle log (what shipped this session)
tools/         native build launchers + assembly server (NO host-language tools)
                serve.sh      -> builds + runs assembly/sakum_webhook.s (POST /update)
                sakum.sh      -> builds + runs tools/sakum.s (native Sakum CLI)
                build_trackers.sh / build_app.sh -> compile assembly targets
SAKUM_LANG.md  design doctrine (DO / DON'T / roadmap)
SAKUM_HINGLISH.md canonical Hinglish keyword glossary (single source)
EXTENSIONS.sakdoc canonical file-type extension registry (single source of truth)
docs/EXTENSIONS.sakdoc.pdf rendered extension reference
tools/sakum_ext.py registrar/dispatcher that honors the extension scheme
tools/make_ext_pdf.py regenerates the PDF + .tex from EXTENSIONS.sakdoc
sakum_lang.sakproj project config declaring the extension map
sakum_knowledge.sakpkg knowledge package manifest

```

Status

Machine-level core (phase 2 of roadmap reached: the language bootstraps itself in assembly). Additional ISA back ends (aarch64 NEON, RISC-V RVV) and the live quantum backend are in progress — see `SAKUM_LANG.md` §4.

Self-updater bot (local, self-hosting, machine-level only)

A small agent that keeps the Sakum core current with upstream programming-language developments and rewrites its own code through the `self` engine. It runs as **raw assembly** — no Python, no bash script performing project logic. Every patch it emits must compile to raw assembly (`assembly/sakum_*.s`).

```

# run the from-scratch assembly webhook server (POST /update + GET /status)
bash tools/serve.sh 8080 600          # port 8080, pulse every 600s
# POST /update                        -> publishes webhook.update on the naadi bus -> cycle
# GET /status                         -> dumps memory.md
# GET /nerve                          -> naadi bus channels + last signals
curl -X POST http://127.0.0.1:8080/update

# OS-native timer (macOS launchd) - pulses even with the server stopped

```

```
cp tools/com.sakum.bot.plist ~/Library/LaunchAgents/
launchctl load ~/Library/LaunchAgents/com.sakum.bot.plist
launchctl start com.sakum.bot
```

The bot obeys `learn.md`, records outcomes in `memory.md`, writes self-patches to `self/patches/patch_<ts>.json` (schema in `SAKUM_LANG.md §1.7`), recompiles `assembly/sakum_*.s`, and on any compile failure rolls the patch back and logs a mistake to the binary-hash ledger (`query_logs/type_1_memory.jsonl`). See `tools/README.md` for the full contract.

The bot stays **keep-alive and silently learning**: a macOS launchd timer (`tools/com.sakum.bot.plist`, `StartInterval=600`, `KeepAlive` on failure) runs a cycle forever. Each cycle the bot **generates real, compilable library functions** in `assembly/sakum_lib_*.s` + `examples/lib_*.sak`, recompiles the whole core, and **rolls back + self-heals** any patch that fails to build. The `brahma` crawler (`assembly/sakum_bramann.s`) quantum-learns across spheres each pulse, logging research to `research.md` and improvements to `upgrade.md` / `update.md`. A from-scratch assembly webhook receiver (`assembly/sakum_webhook.s`) also answers `POST /update` directly at the machine level, and the bot authors its own web stack in Sakum (`examples/bot_self.sak`).

Activate (always-on)

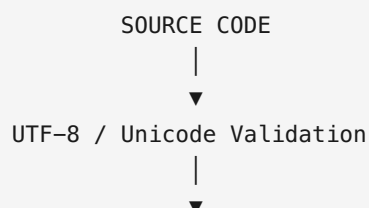
```
# macOS launchd: pulses every 10 min, auto-relaunches on failure
cp tools/com.sakum.bot.plist ~/Library/LaunchAgents/
launchctl load ~/Library/LaunchAgents/com.sakum.bot.plist
launchctl start com.sakum.bot

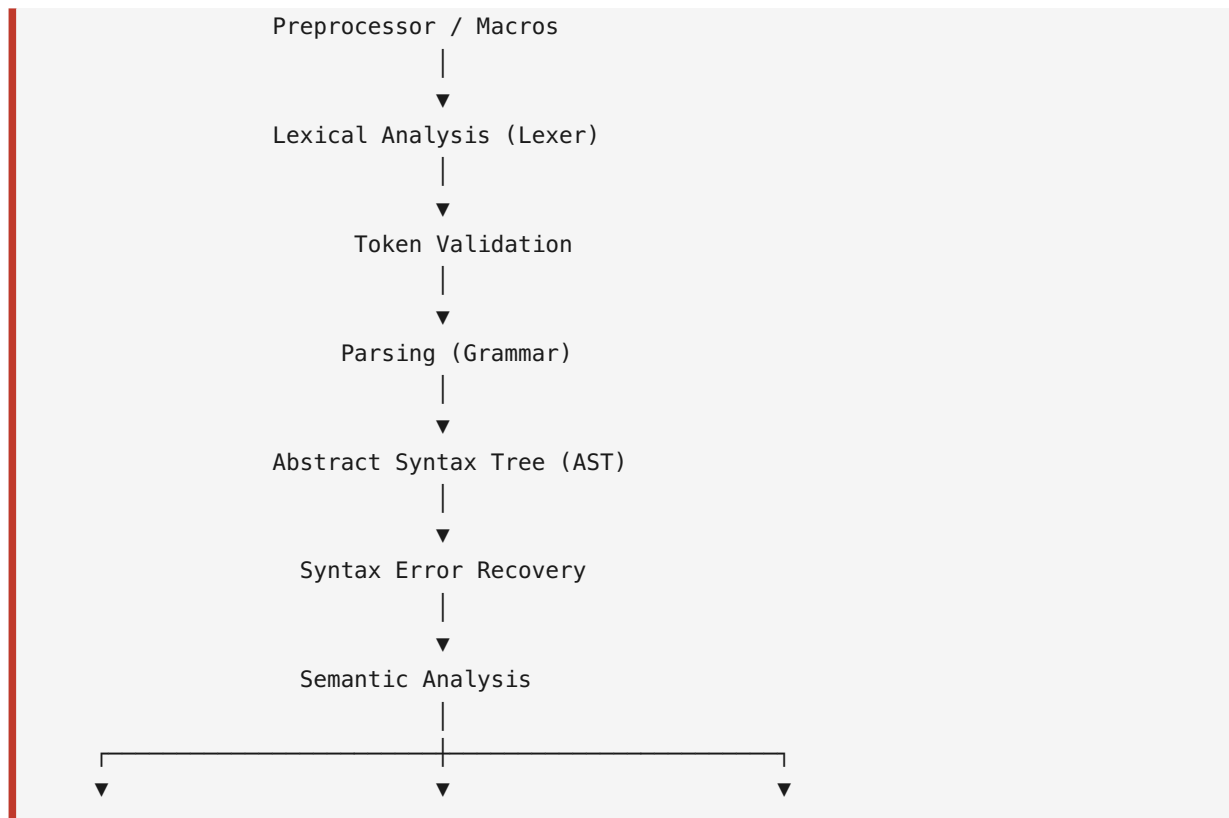
# or run the local assembly webhook + timer server
bash tools/serve.sh 8080 600
```

To stop: `launchctl unload ~/Library/LaunchAgents/com.sakum.bot.plist`

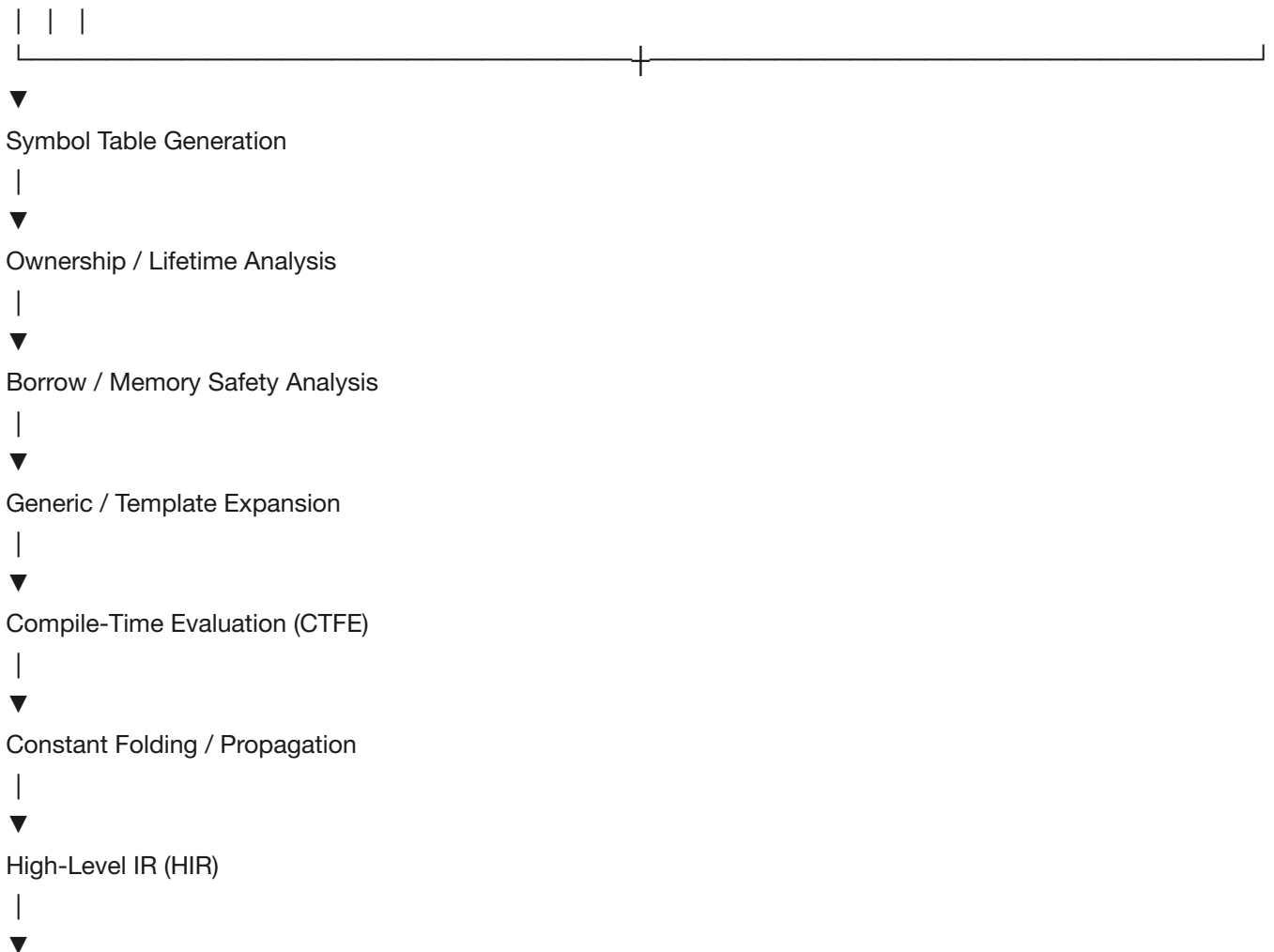
both the method will work user can call any to any place

Method1





Type Checking Scope Resolution Name Resolution



HIR Validation & Verification

|



High-Level Optimizations

|



Mid-Level IR (MIR / SSA)

|



Control Flow Graph (CFG) Construction

|



Data Flow & Dependency Analysis

|



Alias & Escape Analysis

|



Memory Optimization Passes

|



Security Validation Passes

|



Dead Code Elimination / Inlining

|



Loop & Vectorization Optimizations

|



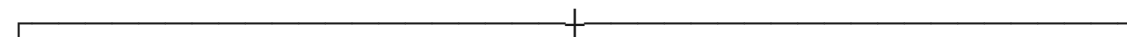
Low-Level IR (LIR)

|



Backend Capability Check

|



Native Backend VM Backend WASM Backend

| | |



Machine IR VM Bytecode WASM IR

| | |



Register Allocation Bytecode Verify WASM Verify

| | |



Instruction Scheduling VM Optimization WASM Optimize



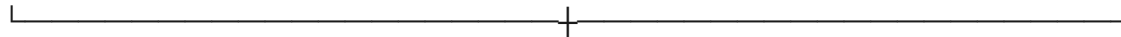
Assembly Sanskrit Bytecode .wasm Module



Assembler VM Package Builder WASM Linker



Object Files VM Executable WASM Binary



Universal Linker



Library Dependency Resolver



Symbol Resolution & Relocation



Executable / Shared Library



Binary Validation & Verification



Binary Size Optimization (Optional)



Debug Symbol Generation (Optional)



Package / Installer Generation



Digital Code Signing



Production Security Verification



Operating System Loader

|
▼
Runtime Initialization (CRT)
|
▼
Memory Layout Creation
|
▼
CPU Fetch → Decode → Execute

Method 2

SOURCE CODE (.s)

|
▼
1. Text Encoding (UTF-8/ASCII)

|
▼
2. Lexical Analysis (Lexer)

|
▼
Tokens

|
▼
3. Parsing (Parser)

|
▼
Abstract Syntax Tree (AST)

|
▼
4. Semantic Analysis

|
▼
Typed AST / Symbol Tables

|
▼
5. Intermediate Representation (IR)

|
▼
6. Optimizations

|
▼
Optimized IR
|



7. Code Generation



Assembly (.s)



8. Assembler



Object File (.o/.obj)



9. Linker + Libraries (.a/.lib/.so/.dll)



Executable (.exe/.out/.elf)



10. Loader (Operating System)



Memory + Stack + Heap + Shared Libraries



CPU Fetch → Decode → Execute

Implement complete Latex in assembly , x86 x64 , asm , risc v , all architecture support

next task - increase its knowledge and increase its library function everything in assembly , x86 x64 , asm , risc v , all architecture support

next task always check where it left , fix it with available method send it to khoj.md which is still need to finish task

create own dictionary with language learning , start with english , hinglish , hindi, maths, physics, chemistry, and all the things

create its own encryption key name APRA ()

use <https://tour.gleam.run/basics/hello-world/> language like learning tour

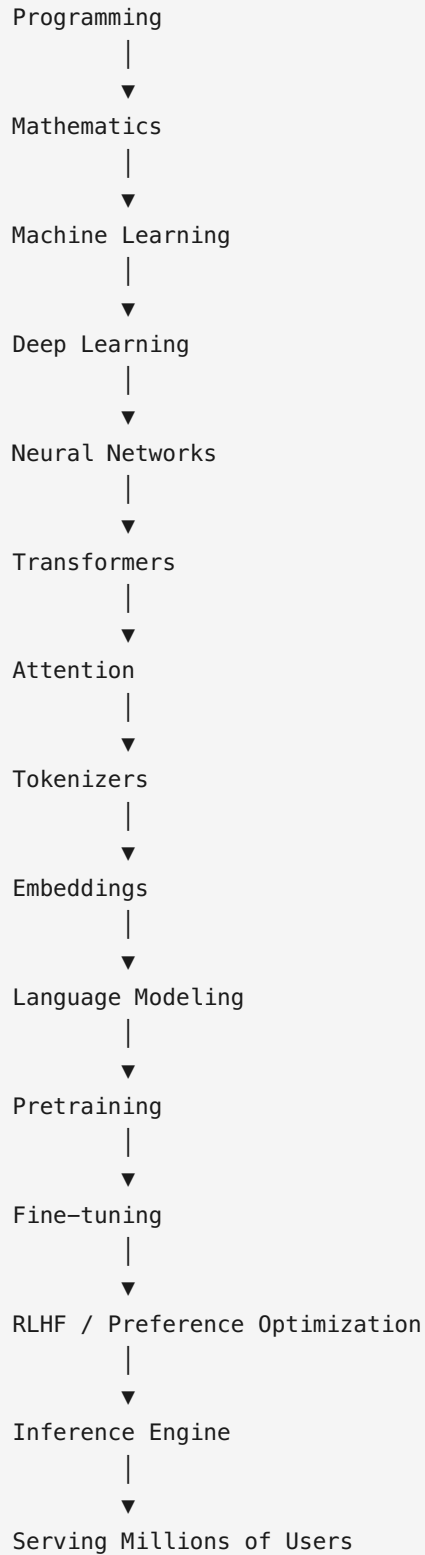
create complete website copy of gleam and rust language web pages style look & feel

Learn this and create machine code and library function for each

If your goal is to build something on the level of Claude, ChatGPT, GLM, Gemma, or Qwen, you're aiming to learn one of the most complex engineering disciplines today. It combines mathematics, machine learning, distributed systems, software engineering, and infrastructure.

The good news is that you don't have to learn it all at once. You can follow the same path that OpenAI, Anthropic, Google, Meta, and Zhipu AI engineers follow, starting from the fundamentals.

Complete Roadmap (0 → GPT/Claude-Level)



Phase 1 – Foundations

Before AI comes programming.

Learn (in Sakum Lang or your host language of choice — the Sakum project itself contains no foreign host-language code, per SAKUM_LANG.md §2):

- Variables
- Functions
- Loops
- Classes
- File handling
- JSON
- APIs
- NumPy
- Pandas

Project

Chatbot
Calculator
Weather API
Web Scraper

Time

2 weeks

Phase 2 – Mathematics

Most people skip this.

Don't.

Learn

Linear Algebra

Vectors

Matrices

Matrix multiplication

Transpose

Inverse
Eigenvectors

Calculus

Derivatives
Gradients
Partial derivatives
Chain Rule

Probability

Mean
Variance
Normal Distribution
Bayes Rule
Maximum Likelihood

Optimization

Gradient Descent
Adam
Momentum
Learning Rate

Phase 3 – Machine Learning

Learn

Regression
Classification
Decision Trees
Random Forest

KNN

Naive Bayes

SVM

Clustering

PCA

Libraries

Scikit Learn

Projects

Spam detector

House price prediction

Customer segmentation

Phase 4 – Neural Networks

Understand

Neuron

Input

Weights

Bias

Activation

Output

Build

One neuron

Multiple neurons

MLP

Backpropagation

Implement everything using NumPy.

No PyTorch yet.

Phase 5 – Deep Learning

Libraries

PyTorch

Learn

Tensor

Autograd

Dataset

DataLoader

GPU

CUDA

Loss Functions

Optimizers

Projects

MNIST

CIFAR

Cats vs Dogs

Phase 6 – NLP

Learn

One Hot Encoding

Word2Vec

FastText

GloVe

Embeddings

Sentence Embeddings

Tokenization

BPE

SentencePiece

WordPiece

Unigram

Projects

Sentiment analysis

Text classification

Phase 7 – Transformer

This changes everything.

Read

Attention Is All You Need

Learn

Encoder

Decoder

Attention

Multi Head Attention

Residual

LayerNorm

Feed Forward

Positional Encoding

Build

Transformer from scratch

using PyTorch.

Phase 8 – LLM

Understand

Next Token Prediction

Example

The sky is ____

↓

blue

Learn

Cross Entropy

Teacher Forcing

Context Window

Perplexity

Vocabulary

Phase 9 – Tokenizer

Build

BPE Tokenizer

Learn

Vocabulary

Merge Rules

Compression

Subwords

Phase 10 – Build GPT

Now build

- Embedding
- Transformer Blocks
- Attention
- MLP
- Residual
- Output Layer

Train

- Tiny Shakespeare

This is exactly how almost everyone starts.

Phase 11 – Scale Up

Train on

Wikipedia

Books

GitHub

Common Crawl

Arxiv

StackOverflow

Research papers

Millions of documents.

Phase 12 – Distributed Training

Learn

DDP

DeepSpeed

FSDP

Megatron-LM

Pipeline Parallelism

Tensor Parallelism

Checkpointing

Hardware

8 GPUs

64 GPUs

512 GPUs

Phase 13 – Instruction Tuning

Teach the model

Question

↓

Answer

Use

Alpaca

ShareGPT

OpenAssistant

UltraChat

Phase 14 – RLHF

Claude

GPT

Gemini

GLM

all use something similar.

Pipeline

Pretraining

↓

Supervised Fine-Tuning

↓

Reward Model

↓

Preference Optimization

↓

RLHF / DP0

Phase 15 – Inference Engine

Learn

KV Cache

Speculative Decoding

Paged Attention

Flash Attention

Continuous Batching

Sampling

Top-K

Top-P

Temperature

Frameworks

vLLM

llama.cpp

TensorRT-LLM

SGLang

Phase 16 – Quantization

Learn

FP32

FP16

BF16

INT8

INT4

GGUF

GPTQ

AWQ

EXL2

Phase 17 – Build ChatGPT

Components

User

↓

Web UI

↓

API

↓

Inference Server

↓

LLM

↓

Vector DB

↓

Tools

↓

Memory

↓

Search

↓

Agent

Phase 18 – Multimodal AI

Add

Images

OCR

Video

Audio

Speech

Vision

Projects

Image captioning

Document QA

Voice assistant

Phase 19 – AI Agents

Learn
Planning
Memory
Tool use
Code execution
Browser automation
Research agents
Multi-agent systems

Phase 20 – Production

Deploy
Docker
Kubernetes
Ray
Monitoring
Logging
Autoscaling
Security
Evaluation

Books

I recommend reading these in roughly this order:

1. Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow
 2. Deep Learning
 3. Natural Language Processing with Transformers
 4. Build a Large Language Model (From Scratch)
-

Open-source repositories to study

These projects are excellent references for different stages of LLM development:

- [llama.cpp](#) (efficient local inference)
- [nanoGPT](#) (minimal GPT training)
- [litgpt](#) (modern LLM training)
- [Megatron-LM](#) (large-scale distributed training)
- [DeepSpeed](#) (training optimization)
- [vLLM](#) (high-throughput inference)
- [Hugging Face Transformers](#) (pretrained models and tooling)

A practical course plan

Since you've mentioned you want to build your own open-source AI system and you're working on an Apple M1 machine, I'd suggest we turn this into a hands-on course rather than just reading theory.

The progression would look like this:

1. Programming foundations for AI (practical)
2. Math with visual examples
3. Build a neuron from scratch
4. Build a neural network from first principles
5. Implement backpropagation yourself
6. Build a tokenizer
7. Build attention
8. Build a Transformer
9. Train a mini-GPT on a small text corpus
10. Add instruction tuning
11. Add retrieval, tools, and agent capabilities
12. Optimize it for local inference with quantization

By the end, you'll understand not just how to *use* models like ChatGPT or GLM, but how the core architecture is built, trained, optimized, and deployed.

create it own wireshark and nmap capabilities

check if everything is functional , keep the website updated , with new thing learn by this language

create it own AI from scratch with auto scale capabilities , and can be broken into chunk according to categories , sub categories and use it only necessary , so that it will work on limit ram or without gpu as well , also auto scale means it will aware of space of ram and hard drive and gpu and adjust itself without memory leak and update itself .

create interactive terminal colorful option like cmd or other beautiful terminals
